PYTHON ACTIVITY-BASED LEARNING ASSIGNMENT

Case Study: Student Timetable Conflict Detection

This program validates a proposed student timetable by checking overlapping class periods, room availability, and schedule conflicts. Course schedules are represented using strings and tuples.

1. Problem Statement

Create a Python program to check whether a proposed student timetable is valid. Each course contains a course name, day, start time, end time, and room. The program must detect overlapping periods for the same student, identify when a room is already occupied by another course, and report whether the timetable is valid.

2. Objectives

• Represent course details using strings and tuples.
• Detect overlapping time periods on the same day.
• Check whether a room is unavailable during a proposed class.
• Validate all courses in a proposed timetable.
• Display clear conflict messages.

3. Assumptions

• Times use 24-hour HH:MM format.
• Each course is represented as a tuple: (course_name, day, start_time, end_time, room).
• A class occupies its start time but not its end time, so 09:00–10:00 and 10:00–11:00 do not overlap.
• The same student's courses cannot overlap on the same day.
• A room cannot host two courses at the same time on the same day.
• Room availability is represented by a list of existing room bookings.
• The timetable is valid only if there are no student-time conflicts, room conflicts, or invalid time ranges.

4. Algorithm

1. Start the program.
2. Store proposed course schedules as tuples.
3. Store existing room bookings as tuples.
4. Convert each time string into minutes after midnight.
5. Check that each course has a valid start and end time.
6. Compare every pair of proposed courses on the same day for overlapping periods.
7. Compare each proposed course with room bookings to identify occupied rooms.
8. Record and display all conflicts.
9. If no conflicts are found, report that the timetable is valid; otherwise report that it is invalid.
10. End the program.

5. Pseudocode

BEGIN
proposed ← list of course tuples
bookings ← list of existing room-booking tuples
conflicts ← empty list
FOR each course c in proposed DO
IF c.start ≥ c.end THEN record invalid time
FOR each other course d after c DO
IF same day AND time intervals overlap THEN
record student timetable conflict
FOR each booking b DO
IF same day AND same room AND intervals overlap THEN
record room unavailable conflict
ENDFOR
IF conflicts is empty THEN DISPLAY valid timetable
ELSE DISPLAY conflicts and invalid timetable
END

6. Flowchart

Start → Read proposed course tuples and room bookings → Validate time ranges → Compare course pairs for student overlap → Compare courses with room bookings → Any conflict? → Yes: display conflict details and mark invalid / No: display timetable valid → End.

7. Python Source Code

```
def to_minutes(time_text):
    """Convert HH:MM into minutes after midnight."""
    hour, minute = map(int, time_text.split(":"))
    if not (0 <= hour <= 23 and 0 <= minute <= 59):
```

```
        raise ValueError("Time must be in 24-hour HH:MM format")
    return hour * 60 + minute

def overlaps(start1, end1, start2, end2):
    return start1 < end2 and start2 < end1

# Tuple format: (course, day, start, end, room)
proposed = [
    ("Python", "Monday", "09:00", "10:00", "Lab-1"),
    ("Mathematics", "Monday", "09:30", "10:30", "Room-101"),
    ("Physics", "Monday", "10:00", "11:00", "Lab-1"),
    ("English", "Tuesday", "11:00", "12:00", "Room-102")
]

# Existing room bookings: (day, start, end, room, booking_name)
room_bookings = [
    ("Monday", "10:00", "11:00", "Lab-1", "Other Class"),
    ("Tuesday", "12:00", "13:00", "Room-102", "Department Meeting")
]

conflicts = []

# Check time ranges and pairwise student timetable overlaps
valid_courses = []
for course in proposed:
    name, day, start, end, room = course
    try:
        start_min = to_minutes(start)
        end_min = to_minutes(end)
        if start_min >= end_min:
            conflicts.append(f"Invalid time range for {name}: {start}-{end}")
        else:
            valid_courses.append(course)
    except ValueError as error:
        conflicts.append(f"{name}: {error}")

for i in range(len(valid_courses)):
    c1 = valid_courses[i]
    for j in range(i + 1, len(valid_courses)):
        c2 = valid_courses[j]
        if c1[1] == c2[1] and overlaps(
            to_minutes(c1[2]), to_minutes(c1[3]),
            to_minutes(c2[2]), to_minutes(c2[3])
        ):
            conflicts.append(
                f"Student schedule conflict: {c1[0]} and {c2[0]} "
                f"overlap on {c1[1]}."
            )

# Check proposed courses against existing room bookings
for course in valid_courses:
    name, day, start, end, room = course
    for booking_day, booking_start, booking_end, booked_room, booking_name in room_bookings:
        if day == booking_day and room == booked_room and overlaps(
            to_minutes(start), to_minutes(end),
            to_minutes(booking_start), to_minutes(booking_end)
        ):
            conflicts.append(
                f"Room conflict: {room} for {name} overlaps with "
                f"{booking_name} on {day}."
            )

print("STUDENT TIMETABLE CONFLICT DETECTION")
print("-" * 65)
for course in proposed:
    print(f"{course[0]:12} | {course[1]:8} | {course[2]}-{course[3]} | {course[4]}")

print("\nVALIDATION RESULT")
if conflicts:
    for item in conflicts:
        print("-", item)
    print("Proposed timetable: INVALID")
else:
    print("No conflicts found.")
    print("Proposed timetable: VALID")
```

8. Test Cases

Test	Situation	Expected result
1	Two courses overlap on Monday	Student schedule conflict reported
2	A course uses Lab-1 during an existing booking	Room conflict reported
3	Classes are 09:00–10:00 and 10:00–11:00	No overlap; times are adjacent
4	End time is earlier than or equal to start time	Invalid time range reported
5	No overlaps and all rooms are free	Timetable is valid

9. Sample Output

```
STUDENT TIMETABLE CONFLICT DETECTION
---------------------------------------------------------------------
Python       | Monday   | 09:00-10:00 | Lab-1
Mathematics  | Monday   | 09:30-10:30 | Room-101
Physics      | Monday   | 10:00-11:00 | Lab-1
English      | Tuesday  | 11:00-12:00 | Room-102

VALIDATION RESULT
- Student schedule conflict: Python and Mathematics overlap on Monday.
- Room conflict: Lab-1 for Physics overlaps with Other Class on Monday.
Proposed timetable: INVALID
```

10. Conclusion

The program checks a proposed timetable for overlapping classes, invalid time ranges, and unavailable rooms. It reports conflicts and determines whether the timetable is valid. This case study demonstrates the use of strings, tuples, lists, loops, functions, conditional statements, and time-interval comparisons in Python.

PYTHON ACTIVITY-BASED LEARNING ASSIGNMENT

Case Study: Railway Fare and Reservation Calculator

This program calculates estimated fares for multiple passengers using distance slabs, travel classes, age-based concessions, reservation fees, and cancellation penalties. The rates used are sample rules for this educational exercise and are not official railway tariffs.

1. Problem Statement

Develop a Python program to calculate railway ticket charges for multiple passengers. The fare depends on the distance travelled and selected travel class. Eligible passengers receive an age-based concession, a reservation fee is added, and a specified cancellation penalty is applied when a ticket is cancelled. The program displays a passenger-wise fare breakdown and a booking summary.

2. Objectives

• Calculate fares based on distance slabs.
• Apply class-based fare multipliers.
• Apply age-based concessions according to sample rules.
• Add reservation fees to ticket costs.
• Calculate cancellation penalties and refund amounts.
• Display a clear summary for multiple passengers.

3. Assumptions

• Sample base fare: ■1.00 per km for distances up to 100 km; ■0.80 per km for the portion from 101–300 km; ■0.60 per km for the portion above 300 km.
• Class multipliers: Sleeper = 1.0, AC 3-Tier = 2.0, AC 2-Tier = 3.0.
• Passengers aged 60 or above receive a 20% concession; children under 12 receive a 50% concession. These are illustrative rules only.
• Reservation fee is ■40 per passenger before concession.
• Cancellation penalty is 25% of the discounted base fare plus reservation fee; refund = total fare – penalty.
• Ages and distances must be positive or zero as appropriate; distance must be greater than 0.

4. Algorithm

1. Start the program.
2. Define distance slabs, class multipliers, concession rules, reservation fee, and cancellation penalty rate.
3. Read the number of passengers.
4. For each passenger, read name, age, distance, travel class, and cancellation status.
5. Calculate base fare using progressive distance slabs.
6. Multiply base fare by the selected class multiplier.
7. Apply the age concession to the class fare, if eligible.
8. Add the reservation fee to get the total fare.
9. If cancelled, calculate the penalty and refund amount.
10. Display each passenger's fare details and final summary.
11. End the program.

5. Pseudocode

```
BEGIN
SET reservation_fee = 40, penalty_rate = 0.25
READ passenger_count
FOR each passenger DO
READ name, age, distance, class, cancelled
base ← progressive_fare(distance)
class_fare ← base × class_multiplier[class]
IF age ≥ 60 THEN concession ← 20%
ELSE IF age < 12 THEN concession ← 50%
ELSE concession ← 0%
discounted_fare ← class_fare – concession
total ← discounted_fare + reservation_fee
IF cancelled THEN penalty ← total × 25%; refund ← total – penalty
ELSE penalty ← 0; refund ← 0
DISPLAY fare details
ENDFOR
END
```

6. Flowchart

Start → Set fare slabs, class multipliers, concession rules, and fees → Read passenger details → Calculate distance-based base fare → Apply class multiplier → Apply age concession → Add reservation fee → Cancelled? If yes, calculate penalty and refund → Display passenger fare breakdown → More passengers? If yes, repeat → Display summary → End.

7. Python Source Code

```
RESERVATION_FEE = 40.0
CANCELLATION_RATE = 0.25

CLASS_MULTIPLIERS = {
    "Sleeper": 1.0,
    "AC 3-Tier": 2.0,
    "AC 2-Tier": 3.0
}

def distance_fare(distance):
    """Calculate base fare using progressive distance slabs."""
    fare = 0.0
    first = min(distance, 100)
    fare += first * 1.00

    second = min(max(distance - 100, 0), 200)
    fare += second * 0.80

    third = max(distance - 300, 0)
    fare += third * 0.60
    return fare

def calculate_ticket(name, age, distance, travel_class, cancelled):
    base = distance_fare(distance)
    class_fare = base * CLASS_MULTIPLIERS[travel_class]

    if age >= 60:
        concession_rate = 0.20
    elif age < 12:
        concession_rate = 0.50
    else:
        concession_rate = 0.0

    concession = class_fare * concession_rate
    discounted_fare = class_fare - concession
    total = discounted_fare + RESERVATION_FEE

    if cancelled:
        penalty = total * CANCELLATION_RATE
        refund = total - penalty
    else:
        penalty = 0.0
        refund = 0.0

    return {
        "name": name, "class": travel_class, "distance": distance,
        "base": base, "class_fare": class_fare, "concession": concession,
        "total": total, "cancelled": cancelled,
        "penalty": penalty, "refund": refund
    }

passengers = [
    ("Anita", 25, 150, "Sleeper", False),
    ("Ramesh", 65, 350, "AC 3-Tier", False),
    ("Kiran", 10, 80, "Sleeper", True),
    ("Meera", 35, 250, "AC 2-Tier", False)
]

print("RAILWAY FARE AND RESERVATION CALCULATOR")
print("Sample educational fare rules; not official railway prices.")
print("-" * 76)

total_fares = 0.0
total_penalties = 0.0
total_refunds = 0.0

for data in passengers:
    name, age, distance, travel_class, cancelled = data

    if age < 0 or distance <= 0 or travel_class not in CLASS_MULTIPLIERS:
        print(f"Invalid passenger details for {name}; skipped.")
        continue

    result = calculate_ticket(name, age, distance, travel_class, cancelled)
    print(f'Passenger: {result["name"]} | Age: {age} | Distance: {distance} km')
    print(f'Class: {result["class"]} | Base fare: ■{result["base"]:.2f}')
    print(f'Class fare: ■{result["class_fare"]:.2f} | Concession: ■{result["concession"]:.2f}')
    print(f'Reservation fee: ■{RESERVATION_FEE:.2f} | Total fare: ■{result["total"]:.2f}')

    if cancelled:
        print(f'Cancelled | Penalty: ■{result["penalty"]:.2f} | Refund: ■{result["refund"]:.2f}')
        total_penalties += result["penalty"]
        total_refunds += result["refund"]
```

```
    else:
        total_fares += result["total"]
        print("Status: Confirmed")
    print("-" * 76)

print(f"Total confirmed fares: ■{total_fares:.2f}")
print(f"Total cancellation penalties: ■{total_penalties:.2f}")
print(f"Total refunds due: ■{total_refunds:.2f}")
```

8. Test Cases

Test	Input	Expected result
1	150 km, Sleeper, age 25	Base ■140; class fare ■140; total ■180
2	350 km, AC 3-Tier, age 65	Base ■290; class fare ■580; 20% concession ■116; total ■504
3	80 km, Sleeper, age 10, cancelled	Base ■80; 50% concession ■40; total ■80; penalty ■20; refund ■60
4	250 km, AC 2-Tier, age 35	Base ■220; class fare ■660; total ■700
5	Distance 0 km	Invalid input; passenger is skipped

9. Sample Output

```
RAILWAY FARE AND RESERVATION CALCULATOR
Sample educational fare rules; not official railway prices.
----------------------------------------------------------------------------
Passenger: Anita | Age: 25 | Distance: 150 km
Class: Sleeper | Base fare: ■140.00
Class fare: ■140.00 | Concession: ■0.00
Reservation fee: ■40.00 | Total fare: ■180.00
Status: Confirmed
----------------------------------------------------------------------------
Passenger: Ramesh | Age: 65 | Distance: 350 km
Class: AC 3-Tier | Base fare: ■290.00
Class fare: ■580.00 | Concession: ■116.00
Reservation fee: ■40.00 | Total fare: ■504.00
Status: Confirmed
----------------------------------------------------------------------------
Passenger: Kiran | Age: 10 | Distance: 80 km
Class: Sleeper | Base fare: ■80.00
Class fare: ■80.00 | Concession: ■40.00
Reservation fee: ■40.00 | Total fare: ■80.00
Cancelled | Penalty: ■20.00 | Refund: ■60.00
----------------------------------------------------------------------------
Passenger: Meera | Age: 35 | Distance: 250 km
Class: AC 2-Tier | Base fare: ■220.00
Class fare: ■660.00 | Concession: ■0.00
Reservation fee: ■40.00 | Total fare: ■700.00
Status: Confirmed
----------------------------------------------------------------------------
Total confirmed fares: ■1384.00
Total cancellation penalties: ■20.00
Total refunds due: ■60.00
```

10. Conclusion

The program calculates passenger fares using progressive distance slabs, class multipliers, sample age concessions, reservation fees, and cancellation penalties. It produces passenger-wise fare details and a summary of confirmed fares, penalties, and refunds. This case study demonstrates functions, lists, dictionaries, loops, conditionals, and arithmetic in Python. All fare rules are illustrative and should not be treated as official railway policy.

PYTHON ACTIVITY-BASED LEARNING ASSIGNMENT

Case Study: Hackathon Team Formation and Ranking

This program validates hackathon team sizes, analyses student skill scores, totals each team's scores across multiple rounds, and ranks valid teams. Ties are resolved using predefined rules.

1. Problem Statement

Develop a Python program to manage hackathon teams. The program must check whether each team meets the allowed size, calculate team totals from round-wise scores, and generate rankings. If teams have the same total score, use the highest final-round score as the first tie-breaker, then the highest earlier-round score, and finally the team name in alphabetical order.

2. Objectives

• Validate team sizes against the allowed minimum and maximum.
• Store student names and their individual skill scores.
• Aggregate team scores across multiple hackathon rounds.
• Rank valid teams from highest to lowest score.
• Resolve ties using clearly defined tie-break rules.

3. Assumptions

• Each team must contain 2 to 4 students, inclusive.
• Each student has a skill score from 0 to 100.
• Team round scores are non-negative whole numbers.
• Only teams with valid sizes are ranked.
• Ranking rules: total score (descending), final-round score (descending), first-round score (descending), then team name (A–Z).
• Scores are sample data for educational use.

4. Algorithm

1. Start the program.
2. Set the minimum team size to 2 and maximum to 4.
3. Store each team's student list, student skill scores, and round-wise team scores.
4. Check each team's size.
5. Mark teams outside the allowed size as invalid and exclude them from ranking.
6. For each valid team, calculate the sum of its round scores.
7. Sort valid teams by total score, final-round score, first-round score, and team name according to the tie-break rules.
8. Display each team's size, total score, and rank.
9. End the program.

5. Pseudocode

BEGIN
min_size ← 2; max_size ← 4
valid_teams ← empty list
FOR each team DO
IF min_size ≤ team_size ≤ max_size THEN
total ← sum(round_scores)
add team with total to valid_teams
ELSE
print invalid team size
ENDIF
ENDFOR
SORT valid_teams BY total DESC, final_round DESC,
first_round DESC, team_name ASC
DISPLAY ranked valid_teams
END

6. Flowchart

Start → Set team-size limits → Read team and round data → Check team size → Valid? If no, flag team as invalid; if yes, calculate total score and add to valid teams → Sort by total score and tie-break rules → Display rankings and invalid teams → End.

7. Python Source Code

```
MIN_SIZE = 2
MAX_SIZE = 4

# Team: name, student skill scores, round-wise team scores
```

```
teams = [
    {"name": "CodeCrafters", "students": {"Asha": 88, "Ravi": 82},
     "rounds": [85, 90, 92]},
    {"name": "ByteBuilders", "students": {"Meena": 91, "Kiran": 86, "Dev": 80},
     "rounds": [90, 88, 89]},
    {"name": "PixelPioneers", "students": {"Isha": 84, "Arun": 87},
     "rounds": [85, 90, 92]},
    {"name": "LogicLegends", "students": {"Nikhil": 78},
     "rounds": [95, 94, 96]},  # Invalid: only one student
    {"name": "DataDynamos", "students": {"Sara": 90, "Vikram": 85, "Tia": 88,
     "Om": 81, "Leah": 79}, "rounds": [92, 91, 90]}  # Invalid: five students
]

valid_teams = []
print("HACKATHON TEAM VALIDATION")
print("-" * 68)

for team in teams:
    size = len(team["students"])
    if not MIN_SIZE <= size <= MAX_SIZE:
        print(f'{team["name"]}: INVALID team size ({size}); allowed: {MIN_SIZE}-{MAX_SIZE}')
        continue

    total = sum(team["rounds"])
    team["total"] = total
    team["size"] = size
    valid_teams.append(team)
    print(f'{team["name"]}: VALID, {size} students, total score = {total}')

# Tie-breakers: total score, final round, first round, team name
valid_teams.sort(
    key=lambda t: (-t["total"], -t["rounds"][-1],
                   -t["rounds"][0], t["name"])
)

print("\nFINAL RANKING")
print("-" * 68)
for rank, team in enumerate(valid_teams, start=1):
    print(f'{rank}. {team["name"]} | Size: {team["size"]} '
          f'| Rounds: {team["rounds"]} | Total: {team["total"]}')

print("\nTie-break rule: total score > final-round score > first-round score > name.")
```

8. Test Cases

Test	Input condition	Expected result
1	Team has 2 students	Valid team; include in ranking
2	Team has 4 students	Valid team; include in ranking
3	Team has 1 student	Invalid team size; exclude from ranking
4	Team has 5 students	Invalid team size; exclude from ranking
5	Equal totals and equal tie-break scores	Compare team names alphabetically
6	Different total scores	Higher total ranks first

9. Sample Output

```
HACKATHON TEAM VALIDATION
--------------------------------------------------------------------
CodeCrafters: VALID, 2 students, total score = 267
ByteBuilders: VALID, 3 students, total score = 267
PixelPioneers: VALID, 2 students, total score = 267
LogicLegends: INVALID team size (1); allowed: 2-4
DataDynamos: INVALID team size (5); allowed: 2-4

FINAL RANKING
--------------------------------------------------------------------
1. ByteBuilders | Size: 3 | Rounds: [90, 88, 89] | Total: 267
2. CodeCrafters | Size: 2 | Rounds: [85, 90, 92] | Total: 267
3. PixelPioneers | Size: 2 | Rounds: [85, 90, 92] | Total: 267

Tie-break rule: total score > final-round score > first-round score > name.
```

10. Conclusion

The program validates team composition, calculates cumulative scores across hackathon rounds, and ranks eligible teams consistently. It demonstrates Python dictionaries, lists, loops, conditional statements, sorting, and tie-breaking logic. Invalid team sizes are reported and excluded from the final ranking.

PYTHON ACTIVITY-BASED LEARNING ASSIGNMENT

Case Study: ATM Menu-Driven Banking Simulation

Develop a menu-driven Python program that allows a user to deposit money, withdraw money, check the account balance, and view a transaction summary. The program repeats until the user selects Exit and validates each operation.

1. Problem Statement

Create a simple ATM simulation using Python. The program must repeatedly display a menu for deposit, withdrawal, balance inquiry, transaction summary, and exit. It must validate transaction amounts, prevent withdrawals greater than the available balance, and display a summary of completed transactions.

2. Objectives

• Implement a repeated menu using a loop.
• Perform deposits and withdrawals.
• Display the current account balance.
• Store and summarize completed transactions.
• Validate inputs and prevent invalid withdrawals.

3. Assumptions

• The account starts with a balance of ■5,000.
• The account holder is already authenticated; PIN verification is outside this simulation.
• Deposit and withdrawal amounts must be positive numbers.
• A withdrawal cannot exceed the current balance.
• Only successful deposits and withdrawals are recorded in the transaction summary.
• The program ends when the user selects option 5.

4. Algorithm

1. Start and set the initial balance to ■5,000.
2. Create an empty list to store transactions.
3. Display the ATM menu.
4. Read the user's choice.
5. For Deposit, validate the amount and add it to the balance.
6. For Withdrawal, validate the amount and check sufficient balance before subtracting it.
7. For Balance Inquiry, display the current balance.
8. For Transaction Summary, display all successful transactions and totals.
9. Repeat the menu until Exit is selected.
10. End the program.

5. Pseudocode

```
BEGIN
balance ← 5000
transactions ← empty list
REPEAT
DISPLAY menu
INPUT choice
IF choice = 1 THEN
INPUT amount
IF amount > 0 THEN balance ← balance + amount; record deposit
ELSE DISPLAY invalid amount
ELSE IF choice = 2 THEN
INPUT amount
IF amount <= 0 THEN DISPLAY invalid amount
ELSE IF amount > balance THEN DISPLAY insufficient balance
ELSE balance ← balance - amount; record withdrawal
ELSE IF choice = 3 THEN DISPLAY balance
ELSE IF choice = 4 THEN DISPLAY transaction summary
ELSE IF choice = 5 THEN DISPLAY goodbye
ELSE DISPLAY invalid choice
```

UNTIL choice = 5
END

6. Flowchart

Start → Set balance = ■5,000 and empty transaction list → Display menu → Read choice → Deposit / Withdraw / Balance Inquiry / Transaction Summary / Exit → Validate amount and balance where needed → Update balance and record successful transaction → Display result → If not Exit, return to menu → End.

7. Python Source Code

```
balance = 5000.0
transactions = []

while True:
    print("\n===== ATM MENU =====")
    print("1. Deposit")
    print("2. Withdraw")
    print("3. Balance Inquiry")
    print("4. Transaction Summary")
    print("5. Exit")

    choice = input("Enter your choice (1-5): ").strip()

    if choice == "1":
        try:
            amount = float(input("Enter deposit amount: ■"))
            if amount <= 0:
                print("Invalid amount. Enter a positive value.")
            else:
                balance += amount
                transactions.append(("Deposit", amount, balance))
                print(f"Deposit successful. Balance: ■{balance:.2f}")
        except ValueError:
            print("Invalid input. Enter a numeric amount.")

    elif choice == "2":
        try:
            amount = float(input("Enter withdrawal amount: ■"))
            if amount <= 0:
                print("Invalid amount. Enter a positive value.")
            elif amount > balance:
                print("Transaction failed: insufficient balance.")
            else:
                balance -= amount
                transactions.append(("Withdrawal", amount, balance))
                print(f"Withdrawal successful. Balance: ■{balance:.2f}")
        except ValueError:
            print("Invalid input. Enter a numeric amount.")

    elif choice == "3":
        print(f"Current balance: ■{balance:.2f}")

    elif choice == "4":
        print("\n===== TRANSACTION SUMMARY =====")
        if not transactions:
            print("No successful transactions yet.")
        else:
            total_deposits = 0
            total_withdrawals = 0
            for number, item in enumerate(transactions, start=1):
                kind, amount, remaining = item
                print(f"{number}. {kind}: ■{amount:.2f} | Balance after: ■{remaining:.2f}")
                if kind == "Deposit":
                    total_deposits += amount
                else:
                    total_withdrawals += amount
            print(f"Total deposits: ■{total_deposits:.2f}")
            print(f"Total withdrawals: ■{total_withdrawals:.2f}")
            print(f"Current balance: ■{balance:.2f}")

    elif choice == "5":
        print("Thank you for using the ATM simulation.")
        break

    else:
        print("Invalid choice. Please select from 1 to 5.")
```

8. Test Cases

Test	Input / Operation	Expected Result
1	Deposit ■1,000	Success; balance becomes ■6,000
2	Withdraw ■1,500 from ■6,000	Success; balance becomes ■4,500
3	Withdraw ■5,000 from ■4,500	Rejected; insufficient balance
4	Deposit ■0 or a negative amount	Rejected; positive amount required
5	Balance inquiry	Displays current balance
6	Transaction summary	Lists successful transactions and totals
7	Menu choice 9	Displays invalid choice message

9. Sample Output

```
===== ATM MENU =====
1. Deposit
2. Withdraw
3. Balance Inquiry
4. Transaction Summary
5. Exit
Enter your choice (1-5): 1
Enter deposit amount: ■1000
Deposit successful. Balance: ■6000.00

===== ATM MENU =====
Enter your choice (1-5): 2
Enter withdrawal amount: ■1500
Withdrawal successful. Balance: ■4500.00

===== ATM MENU =====
Enter your choice (1-5): 3
Current balance: ■4500.00

===== ATM MENU =====
Enter your choice (1-5): 4

===== TRANSACTION SUMMARY =====
1. Deposit: ■1000.00 | Balance after: ■6000.00
2. Withdrawal: ■1500.00 | Balance after: ■4500.00
Total deposits: ■1000.00
Total withdrawals: ■1500.00
Current balance: ■4500.00

===== ATM MENU =====
Enter your choice (1-5): 5
Thank you for using the ATM simulation.
```

10. Conclusion

The ATM simulation demonstrates how Python can handle repeated menu operations, update an account balance, validate user inputs, and maintain a transaction history. It uses variables, loops, conditional statements, exception handling, and lists to model basic banking operations. This is an educational simulation and is not connected to a real bank.

PYTHON ACTIVITY-BASED LEARNING ASSIGNMENT

Case Study: Bus Passenger Occupancy Simulation

This program simulates passenger boarding and alighting at multiple bus stops, calculates the number of passengers after each stop, and flags any capacity violations.

1. Problem Statement

Develop a Python program to simulate passenger movement on a bus route with multiple stops. At each stop, the program receives the number of passengers getting off and the number getting on. It calculates the passenger count after each stop and checks whether the bus exceeds its maximum capacity.

2. Objectives

• Track boarding and alighting at multiple stops.
• Calculate the passenger count after every stop.
• Detect and report capacity violations.
• Display a clear summary of passenger occupancy.

3. Assumptions

• The bus starts with 0 passengers.
• Bus capacity is fixed at 40 passengers.
• Alighting cannot exceed the number of passengers currently on the bus.
• Boarding and alighting values are non-negative whole numbers.
• A capacity violation occurs if occupancy after boarding exceeds 40 passengers. The program reports the violation and continues the simulation for demonstration.

4. Algorithm

1. Start the program.
2. Set bus capacity to 40 and passenger count to 0.
3. Store each stop's name, alighting count, and boarding count.
4. For every stop, subtract alighting passengers from the current count.
5. Add boarding passengers to the updated count.
6. If the count is greater than capacity, flag a capacity violation; otherwise report occupancy as normal.
7. Display occupancy and status for every stop.
8. End the program.

5. Pseudocode

```
BEGIN
capacity ← 40
passengers ← 0
stops ← list of stop records
FOR each stop in stops DO
IF alighting > passengers THEN
PRINT invalid alighting count
CONTINUE to next stop
ENDIF
passengers ← passengers - alighting
passengers ← passengers + boarding
IF passengers > capacity THEN
status ← "CAPACITY VIOLATION"
ELSE
status ← "Within capacity"
ENDIF
PRINT stop, passengers, status
ENDFOR
```

END

6. Flowchart

Start → Set capacity = 40 and passengers = 0 → Read stop data → For each stop: check alighting ≤ current passengers → Update passengers = passengers – alighting + boarding → Is passengers > capacity? → Yes: flag capacity violation / No: within capacity → Display result → More stops? → Yes: next stop / No: End.

7. Python Source Code

```
capacity = 40
passengers = 0

# (stop name, passengers getting off, passengers getting on)
stops = [
    ("Central Station", 0, 18),
    ("Market Road", 5, 20),
    ("College Junction", 10, 15),
    ("City Hospital", 12, 8),
    ("Railway Station", 8, 10)
]

print("BUS PASSENGER OCCUPANCY SIMULATION")
print("Bus capacity:", capacity)
print("-" * 62)

for stop, alighting, boarding in stops:
    if alighting > passengers:
        print(f"{stop}: Invalid data - only {passengers} passengers are on the bus, "
              f"but {alighting} are trying to get off.")
        continue

    passengers = passengers - alighting
    passengers = passengers + boarding

    if passengers > capacity:
        status = "CAPACITY VIOLATION"
    else:
        status = "Within capacity"

    print(f"{stop:20} | Off: {alighting:2} | On: {boarding:2} "
          f"| Passengers: {passengers:2} | {status}")

print("-" * 62)
print("Final passenger count:", passengers)
```

8. Test Cases

Test	Alighting	Boarding	Expected result
Central Station	0	18	18 passengers; within capacity
Market Road	5	20	33 passengers; within capacity
College Junction	10	15	38 passengers; within capacity
City Hospital	12	8	34 passengers; within capacity
Railway Station	8	10	36 passengers; within capacity

Additional capacity test: If 35 passengers are already on the bus and 10 more board with nobody getting off, occupancy becomes 45. The program must display CAPACITY VIOLATION.

9. Sample Output

```
BUS PASSENGER OCCUPANCY SIMULATION
Bus capacity: 40
--------------------------------------------------------------
Central Station      | Off:  0 | On: 18 | Passengers: 18 | Within capacity
Market Road          | Off:  5 | On: 20 | Passengers: 33 | Within capacity
College Junction     | Off: 10 | On: 15 | Passengers: 38 | Within capacity
City Hospital        | Off: 12 | On:  8 | Passengers: 34 | Within capacity
```

```
Railway Station      | Off:  8 | On: 10 | Passengers: 36 | Within capacity
------------------------------------------------------------
Final passenger count: 36
```

10. Conclusion

The program successfully simulates passenger boarding and alighting across multiple stops. It calculates occupancy after each stop and identifies when the passenger count exceeds the bus capacity. This demonstrates the use of variables, lists, loops, conditional statements, and formatted output in Python.

CASE STUDY – CAMPUS FESTIVAL REGISTRATION AND FINANCIAL AUDIT

1. Problem Statement

Design a Python program to reconcile campus festival participant registrations, registration fees, sponsorship contributions, and expenses. The program detects duplicate registration IDs, counts each valid registration once, and calculates the festival's final financial balance.

2. Objectives

1. Store participant registration details.

2. Detect duplicate registration IDs.

3. Calculate valid registration-fee income.

4. Add sponsorship contributions to total income.

5. Calculate total festival expenses.

6. Determine the final balance and display a financial audit summary.

3. Assumptions

Each participant has a unique registration ID; repeated IDs are treated as duplicates.

For this sample audit, duplicate registration IDs are not counted a second time.

The registration fee is ■500 per valid participant.

Sponsorship contributions and expenses are stored as lists of amounts in Indian rupees.

Final balance = registration income + sponsorship income – total expenses.

A positive balance is a surplus; a negative balance is a deficit.

4. Algorithm

1. Store participant registration records, sponsorship amounts, and expense amounts.

2. Create a dictionary to keep the first occurrence of each registration ID.

3. If a registration ID appears again, record it as a duplicate.

4. Calculate registration income using unique registrations only.

5. Add sponsorship contributions to find total income.

6. Add all expenses to find total expenditure.

7. Calculate final balance and display duplicate records and the audit summary.

5. Pseudocode

```
START
Store participant registrations, sponsorships, and expenses
Set unique registrations to empty dictionary
Set duplicates to empty list
FOR each participant:
    IF registration ID already exists:
        Add record to duplicates
    ELSE:
        Store participant as a unique registration
Calculate registration income from unique registrations
Calculate sponsorship total and expense total
total income = registration income + sponsorship total
balance = total income - expense total
Display duplicate registrations and financial summary
STOP
```

6. Flowchart

```
START
  |
Store registrations, sponsorships, expenses
  |
Check each registration ID
  |
Duplicate? -- YES --> Record duplicate
```

```
  | NO                |
Keep unique record <---+
  |
Calculate registration income
  |
Add sponsorships and expenses
  |
Calculate final balance
  |
Display audit report
  |
 STOP
```

7. Python Source Code

Campus Festival Registration and Financial Audit participants = [("P101", "Aarav", 500), ("P102", "Diya", 500), ("P103", "Kabir", 500), ("P102", "Diya", 500), # Duplicate ID ("P104", "Meera", 500)] sponsorships = [10000, 7500, 5000] expenses = [6000, 4500, 3000, 2500] unique_registrations = {} duplicates = [] for reg_id, name, fee in participants: if reg_id in unique_registrations: duplicates.append((reg_id, name)) else: unique_registrations[reg_id] = (name, fee) registration_income = sum(fee for name, fee in unique_registrations.values()) sponsorship_income = sum(sponsorships) total_expenses = sum(expenses) total_income = registration_income + sponsorship_income final_balance = total_income - total_expenses print("CAMPUS FESTIVAL FINANCIAL AUDIT") print("-" * 40) print("Registrations entered:", len(participants)) print("Valid unique registrations:", len(unique_registrations)) print("Duplicate registrations:", len(duplicates)) for reg_id, name in duplicates: print("Duplicate ID:", reg_id, "| Name:", name) print("-" * 40) print("Registration income: Rs.", registration_income) print("Sponsorship income: Rs.", sponsorship_income) print("Total income: Rs.", total_income) print("Total expenses: Rs.", total_expenses) print("Final balance: Rs.", final_balance) if final_balance > 0: print("Status: Surplus") elif final_balance < 0: print("Status: Deficit") else: print("Status: Break-even")

8. Test Cases

Test Case 1: Five entries contain one repeated ID → 4 unique registrations and 1 duplicate.

Test Case 2: Four unique registrations at ■500 each → registration income = ■2,000.

Test Case 3: Sponsorships ■10,000, ■7,500, ■5,000 → sponsorship total = ■22,500.

Test Case 4: Expenses ■6,000, ■4,500, ■3,000, ■2,500 → expense total = ■16,000.

Test Case 5: Total income ■24,500 and expenses ■16,000 → final balance = ■8,500 surplus.

9. Sample Output

```
CAMPUS FESTIVAL FINANCIAL AUDIT
----------------------------------------
Registrations entered: 5
Valid unique registrations: 4
Duplicate registrations: 1
Duplicate ID: P102 | Name: Diya
----------------------------------------
Registration income: Rs. 2000
Sponsorship income: Rs. 22500
Total income: Rs. 24500
Total expenses: Rs. 16000
Final balance: Rs. 8500
Status: Surplus
```

10. Conclusion

The Campus Festival Registration and Financial Audit program identifies duplicate registrations, counts valid registration fees once, combines sponsorship contributions, totals expenses, and calculates the final financial balance. It demonstrates lists, tuples, dictionaries, loops, conditional statements, sum(), and basic financial reconciliation in Python.

CASE STUDY – STUDENT ACADEMIC RISK CLASSIFICATION

1. Problem Statement

Design a Python program that combines students' marks, attendance percentages, and laboratory scores to classify academic risk, identify multiple simultaneous risk factors, and generate a summary report.

2. Objectives

1. Store student names, marks, attendance, and laboratory scores in lists.

2. Identify low marks, low attendance, and low laboratory scores.

3. Classify students into Low, Medium, and High risk categories.

4. Identify students with more than one risk factor.

5. Generate a summary showing the number of students in each risk category.

3. Assumptions

Marks and laboratory scores are out of 100; attendance is a percentage.

Low marks means marks below 50.

Low attendance means attendance below 75%.

Low laboratory score means lab score below 50.

Two or more risk factors mean High risk; one factor means Medium risk; no factors means Low risk.

These thresholds are example rules for this academic exercise and should be replaced by an institution's official policy when applicable.

4. Algorithm

1. Store student details in parallel lists.

2. Initialize counters for Low, Medium, and High risk.

3. For each student, check marks, attendance, and laboratory score against thresholds.

4. Count the risk factors found.

5. Assign High, Medium, or Low risk based on the number of factors.

6. Display each student's risk category and risk factors.

7. Display the total number of students in each category and identify students with multiple factors.

5. Pseudocode

```
START
Store names, marks, attendance, and lab scores
Set Low, Medium, High counters to zero
FOR each student:
    Set risk_factors to an empty list
    IF marks < 50: add "Low marks"
    IF attendance < 75: add "Low attendance"
    IF lab score < 50: add "Low lab score"
    IF number of risk factors >= 2:
        category = High
    ELSE IF number of risk factors == 1:
        category = Medium
    ELSE:
        category = Low
    Update category counter
    Display student, category, and risk factors
Display category summary and multiple-risk students
STOP
```

6. Flowchart

```
START
  |
Store student lists
  |
Check marks, attendance, and lab score
  |
Count risk factors
  |
2 or more? -- YES --> High Risk
```

```
  | NO
1 factor? --- YES --> Medium Risk
  | NO
Low Risk
  |
Update summary and display results
  |
 STOP
```

7. Python Source Code

Student Academic Risk Classification names = ["Asha", "Bharat", "Charan", "Divya", "Eshan"] marks = [78, 48, 55, 82, 35] attendance = [92, 68, 58, 45, 72] lab_scores = [85, 42, 60, 78, 30] low_risk = 0 medium_risk = 0 high_risk = 0 multiple_risk_students = [] print("STUDENT ACADEMIC RISK REPORT") print("-" * 68) for i in range(len(names)): factors = [] if marks[i] < 50: factors.append("Low marks") if attendance[i] < 75: factors.append("Low attendance") if lab_scores[i] < 50: factors.append("Low lab score") if len(factors) >= 2: category = "High" high_risk += 1 elif len(factors) == 1: category = "Medium" medium_risk += 1 else: category = "Low" low_risk += 1 if len(factors) >= 2: multiple_risk_students.append(names[i]) factor_text = ", ".join(factors) if factors else "None" print("Name:", names[i]) print("Marks:", marks[i], "| Attendance:", attendance[i], "% | Lab:", lab_scores[i]) print("Risk category:", category) print("Risk factors:", factor_text) print("-" * 68) print("SUMMARY") print("Low risk:", low_risk) print("Medium risk:", medium_risk) print("High risk:", high_risk) print("Students with multiple risk factors:", ", ".join(multiple_risk_students) if multiple_risk_students else "None")

8. Test Cases

Test Case 1: Marks 78, attendance 92%, lab 85 → No risk factors → Low risk.

Test Case 2: Marks 48, attendance 68%, lab 42 → Three risk factors → High risk.

Test Case 3: Marks 55, attendance 58%, lab 60 → One risk factor → Medium risk.

Test Case 4: Marks 82, attendance 45%, lab 78 → Low attendance only → Medium risk.

Test Case 5: Marks 35, attendance 72%, lab 30 → Low marks and low lab score → High risk.

9. Sample Output

```
STUDENT ACADEMIC RISK REPORT
--------------------------------------------------------------------
Name: Asha
Marks: 78 | Attendance: 92 % | Lab: 85
Risk category: Low
Risk factors: None
--------------------------------------------------------------------
Name: Bharat
Marks: 48 | Attendance: 68 % | Lab: 42
Risk category: High
Risk factors: Low marks, Low attendance, Low lab score
--------------------------------------------------------------------
Name: Charan
Marks: 55 | Attendance: 58 % | Lab: 60
Risk category: Medium
Risk factors: Low attendance
--------------------------------------------------------------------
Name: Divya
Marks: 82 | Attendance: 45 % | Lab: 78
Risk category: Medium
Risk factors: Low attendance
--------------------------------------------------------------------
Name: Eshan
Marks: 35 | Attendance: 72 % | Lab: 30
Risk category: High
Risk factors: Low marks, Low lab score
--------------------------------------------------------------------
SUMMARY
Low risk: 1
Medium risk: 2
High risk: 2
Students with multiple risk factors: Bharat, Eshan
```

10. Conclusion

The Student Academic Risk Classification program evaluates students using marks, attendance, and laboratory scores. It detects simultaneous risk factors, assigns a risk category, and produces a summary. The program demonstrates lists, loops, conditional statements, counters, and list operations in Python. Risk categories are illustrative and should not replace an institution's official student-support process.

CASE STUDY – FOOD SUPPLY CHAIN STOCK RECONCILIATION

1. Problem Statement

Design a Python program to compare food stock received, distributed, damaged, and remaining across multiple locations. The program calculates expected remaining stock, identifies discrepancies between expected and recorded quantities, and determines replenishment requirements.

2. Objectives

1. Store stock details for several locations.

2. Calculate expected remaining stock from received, distributed, and damaged quantities.

3. Compare expected stock with recorded remaining stock.

4. Identify shortages or excesses as discrepancies.

5. Calculate replenishment quantity needed to reach a target stock level.

6. Display a reconciliation report for all locations.

3. Assumptions

Stock quantities are measured in kilograms (kg).

The example includes Central Store, North Center, and South Center.

Expected remaining = received – distributed – damaged.

Discrepancy = recorded remaining – expected remaining. A negative value means a shortage; a positive value means recorded stock is higher than expected.

Replenishment = target stock – recorded remaining, but never below zero.

All quantities are non-negative whole numbers and refer to the same stock period.

4. Algorithm

1. Store each location's received, distributed, damaged, recorded remaining, and target stock values.

2. For each location, calculate expected remaining stock.

3. Calculate the discrepancy between recorded and expected stock.

4. Calculate replenishment required to reach the target stock.

5. Label the discrepancy as shortage, excess, or no discrepancy.

6. Display all values in a stock reconciliation report.

5. Pseudocode

```
START
Store stock details for each location
FOR each location:
    expected = received - distributed - damaged
    discrepancy = recorded_remaining - expected
    replenishment = target_stock - recorded_remaining
    IF replenishment < 0:
        replenishment = 0
    IF discrepancy < 0:
        status = "Shortage"
    ELSE IF discrepancy > 0:
        status = "Excess"
    ELSE:
        status = "Balanced"
    Display location, expected, discrepancy,
            replenishment, and status
STOP
```

6. Flowchart

```
START
  |
Store stock details
  |
Calculate expected remaining stock
  |
Compare expected with recorded stock
  |
```

```
Calculate discrepancy and replenishment
  |
Classify shortage / excess / balanced
  |
Display reconciliation report
  |
 STOP
```

7. Python Source Code

Food Supply Chain Stock Reconciliation # Values are in kilograms (kg) # location: [received, distributed, damaged, # recorded_remaining, target_stock] stock_data = { "Central Store": [500, 320, 10, 175, 200], "North Center": [200, 130, 5, 60, 100], "South Center": [250, 160, 8, 75, 120] } print("FOOD SUPPLY CHAIN STOCK RECONCILIATION") print("-" * 90) print("Location Received Distributed Damaged Expected Recorded Difference Refill Status") print("-" * 90) for location, data in stock_data.items(): received, distributed, damaged, recorded, target = data expected = received - distributed - damaged difference = recorded - expected refill = max(0, target - recorded) if difference < 0: status = "Shortage" elif difference > 0: status = "Excess" else: status = "Balanced" print(f"{location:<15} {received:<8} {distributed:<11} " f"{damaged:<7} {expected:<8} {recorded:<8} " f"{difference:<10} {refill:<6} {status}") print("-" * 90) print("Difference = recorded stock - expected stock") print("Refill = max(0, target stock - recorded stock)")

8. Test Cases

Test Case	Condition / Example	Expected Result
1	Received 500, distributed 320, damaged 10	Expected remaining = 170 kg
2	Recorded 175, expected 170	Discrepancy = +5 kg (excess)
3	Recorded 60, expected 65	Discrepancy = –5 kg (shortage)
4	Target 100, recorded 60	Replenishment = 40 kg
5	Target 100, recorded 120	Replenishment = 0 kg
6	Recorded stock equals expected stock	Balanced; discrepancy = 0 kg

9. Sample Output

```
FOOD SUPPLY CHAIN STOCK RECONCILIATION
------------------------------------------------------------------------------------------
Location       Received Distributed Damaged Expected Recorded Difference Refill Status
------------------------------------------------------------------------------------------
Central Store   500      320         10      170      175      5          25     Excess
North Center    200      130         5       65       60       -5         40     Shortage
South Center    250      160         8       82       75       -7         45     Shortage
------------------------------------------------------------------------------------------
```

10. Conclusion

The Food Supply Chain Stock Reconciliation program calculates expected remaining stock, compares it with recorded quantities, identifies discrepancies, and estimates replenishment needs at multiple locations. It demonstrates variables, dictionaries, lists, loops, conditional statements, arithmetic operators, and formatted output in Python.

CASE STUDY – SPORTS TOURNAMENT RANKING SYSTEM

1. Problem Statement

Design a Python program to store sports tournament match results and team scores, calculate total points and tie-break values, and produce a final ranking according to a predefined competition policy.

2. Objectives

1. Store team names and match results using lists and tuples.

2. Calculate each team's wins, draws, losses, goals scored, and goals conceded.

3. Award points according to the competition policy.

4. Calculate goal difference as a tie-break value.

5. Rank teams by total points, goal difference, goals scored, and alphabetical order.

6. Display the final tournament table.

3. Assumptions

The tournament has four teams: Falcons, Tigers, Eagles, and Sharks.

Each match result is stored as a tuple: (team1, team2, score1, score2).

A win gives 3 points, a draw gives 1 point to each team, and a loss gives 0 points.

Ranking order: points (descending), goal difference (descending), goals scored (descending), then team name (ascending).

All match scores are valid non-negative integers.

4. Algorithm

1. Store all team names and match results.

2. Initialize statistics for every team.

3. Process each match and update played matches, goals scored, and goals conceded.

4. Award points for wins and draws.

5. Calculate goal difference for every team.

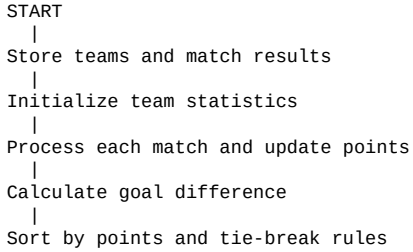
6. Sort teams using the ranking and tie-break rules.

7. Display the final ranking table.

5. Pseudocode

```
START
Store team names and match results
Initialize each team's statistics to zero
FOR each match:
    Update played matches and goal statistics
    IF first score > second score:
        Give 3 points to first team; record win/loss
    ELSE IF second score > first score:
        Give 3 points to second team; record win/loss
    ELSE:
        Give 1 point to each team; record draw
Calculate goal difference for each team
Sort by points, goal difference, goals scored,
then alphabetical team name
Display the final ranking table
STOP
```

6. Flowchart



```
  |
Display final ranking table
  |
 STOP
```

7. Python Source Code

```
# Sports Tournament Ranking System
teams = ["Falcons", "Tigers", "Eagles", "Sharks"]

# (Team 1, Team 2, Team 1 score, Team 2 score)
matches = [
    ("Falcons", "Tigers", 2, 1),
    ("Eagles", "Sharks", 1, 1),
    ("Falcons", "Eagles", 0, 2),
    ("Tigers", "Sharks", 3, 1),
    ("Falcons", "Sharks", 2, 0),
    ("Tigers", "Eagles", 1, 1)
]

# Team statistics
stats = {}
for team in teams:
    stats[team] = {
        "played": 0, "wins": 0, "draws": 0, "losses": 0,
        "goals_for": 0, "goals_against": 0, "points": 0
    }

for team1, team2, score1, score2 in matches:
    stats[team1]["played"] += 1
    stats[team2]["played"] += 1
    stats[team1]["goals_for"] += score1
    stats[team1]["goals_against"] += score2
    stats[team2]["goals_for"] += score2
    stats[team2]["goals_against"] += score1

    if score1 > score2:
        stats[team1]["wins"] += 1
        stats[team1]["points"] += 3
        stats[team2]["losses"] += 1
    elif score2 > score1:
        stats[team2]["wins"] += 1
        stats[team2]["points"] += 3
        stats[team1]["losses"] += 1
    else:
        stats[team1]["draws"] += 1
        stats[team2]["draws"] += 1
        stats[team1]["points"] += 1
        stats[team2]["points"] += 1

for team in teams:
    stats[team]["goal_difference"] = (
        stats[team]["goals_for"] - stats[team]["goals_against"]
    )

ranking = sorted(
    teams,
    key=lambda team: (
        -stats[team]["points"],
        -stats[team]["goal_difference"],
        -stats[team]["goals_for"],
        team
    )
)

print("SPORTS TOURNAMENT FINAL RANKING")
print("-" * 70)
print("Rank Team       P  W  D  L  GF  GA  GD  Points")
print("-" * 70)
for rank, team in enumerate(ranking, start=1):
    x = stats[team]
    print(f"{rank:<4} {team:<10} {x['played']:<2} {x['wins']:<2} "
          f"{x['draws']:<2} {x['losses']:<2} {x['goals_for']:<3} "
          f"{x['goals_against']:<3} {x['goal_difference']:<3} {x['points']}")
print("-" * 70)
print("Policy: Points, goal difference, goals scored, team name.")
```

8. Test Cases

Test Case	Input / Condition	Expected Result
1	Falcons beat Tigers 2–1	Falcons get 3 points; Tigers get 0
2	Eagles and Sharks draw 1–1	Both teams get 1 point

Test Case	Input / Condition	Expected Result
3	Teams have different points	More points ranks higher
4	Points equal; goal difference differs	Higher goal difference ranks higher
5	Points and goal difference equal	More goals scored ranks higher
6	All tie-break values equal	Alphabetical name decides

9. Sample Output

```
SPORTS TOURNAMENT FINAL RANKING
----------------------------------------------------------------------
Rank Team       P  W  D  L  GF  GA  GD  Points
----------------------------------------------------------------------
1    Falcons    3  2  0  1  4   3   1   6
2    Eagles     3  1  2  0  4   2   2   5
3    Tigers     3  1  1  1  5   4   1   4
4    Sharks     3  0  1  2  2   6   -4  1
----------------------------------------------------------------------
Policy: Points, goal difference, goals scored, team name.
```

10. Conclusion

The Sports Tournament Ranking System stores match results, calculates team statistics and points, applies goal difference and other tie-break rules, and displays final rankings. It demonstrates variables, dictionaries, lists, tuples, loops, conditional statements, and sorting in Python.

CASE STUDY – MULTI-EVENT SPORTS SCHEDULING AND SCORING

1. Problem Statement

Design a Python program to represent sports event schedules and team scores using lists and tuples. The program identifies scheduling conflicts, calculates total points for each team, and generates final rankings based on performance.

2. Objectives

1. Store event schedules using lists and tuples.
2. Identify teams or venues scheduled for overlapping events.
3. Store match results and team scores.
4. Aggregate points earned by each team.
5. Rank teams from highest to lowest total points.
6. Display scheduling conflicts and final rankings.

3. Assumptions

Each event is represented as a tuple: (event name, team 1, team 2, venue, start time, end time). Times are represented as integer hours for simple comparison. Two events conflict if they use the same venue at overlapping times, or if a team is scheduled in overlapping events. A win earns 3 points, a draw earns 1 point for each team, and a loss earns 0 points.

4. Algorithm

1. Store event details in a list of tuples.
2. Compare each pair of events to check for overlapping times.
3. Report a conflict if the same team or venue is involved in overlapping events.
4. Store match scores in a list of tuples.
5. Award points based on each match result and add them to the team totals.
6. Sort teams by total points in descending order.
7. Display the conflicts, team points, and final rankings.

5. Pseudocode

```
START
Store event schedules as a list of tuples
FOR each pair of events:
    IF their time intervals overlap:
        IF they share a team or venue:
            Display scheduling conflict
Store match scores
Set each team's points to 0
FOR each match:
    IF team 1 score > team 2 score: award 3 points to team 1
    ELSE IF scores are equal: award 1 point to both teams
    ELSE: award 3 points to team 2
Sort teams by total points, highest first
Display conflicts and final rankings
STOP
```

6. Flowchart

START → Store schedules and scores → Compare event time intervals → Identify shared-team or shared-venue conflicts → Calculate points for each match → Aggregate team totals → Sort teams by points → Display conflicts and final rankings → STOP

7. Python Source Code

```
# Multi-Event Sports Scheduling and Scoring

# Event tuple: (event, team1, team2, venue, start_hour, end_hour)
events = [
    ("Match 1", "Tigers", "Eagles", "Ground A", 9, 11),
    ("Match 2", "Lions", "Wolves", "Ground B", 10, 12),
    ("Match 3", "Tigers", "Lions", "Ground C", 10, 12),
    ("Match 4", "Eagles", "Wolves", "Ground A", 11, 13),
    ("Match 5", "Bears", "Hawks", "Ground B", 13, 15),
]

# Match tuple: (team1, team2, score1, score2)
results = [
    ("Tigers", "Eagles", 2, 1),
    ("Lions", "Wolves", 0, 0),
    ("Tigers", "Lions", 1, 3),
    ("Eagles", "Wolves", 2, 0),
    ("Bears", "Hawks", 4, 2),
]

points = {}
conflicts = []

# Check all pairs of events for overlapping schedules
for i in range(len(events)):
    for j in range(i + 1, len(events)):
        e1 = events[i]
        e2 = events[j]

        # Intervals overlap if each starts before the other ends
        overlap = e1[4] < e2[5] and e2[4] < e1[5]
        if overlap:
            shared_team = (
                e1[1] in (e2[1], e2[2]) or
                e1[2] in (e2[1], e2[2])
            )
            same_venue = e1[3] == e2[3]

            if shared_team or same_venue:
                reason = []
                if shared_team:
                    reason.append("same team")
                if same_venue:
                    reason.append("same venue")
                conflicts.append(
                    (e1[0], e2[0], " and ".join(reason))
                )

# Calculate points from scores
for team1, team2, score1, score2 in results:
    points.setdefault(team1, 0)
    points.setdefault(team2, 0)

    if score1 > score2:
        points[team1] += 3
    elif score1 < score2:
        points[team2] += 3
    else:
        points[team1] += 1
        points[team2] += 1

print("SCHEDULING CONFLICTS")
print("--------------------")
if conflicts:
    for event1, event2, reason in conflicts:
        print(f"{event1} and {event2}: {reason}")
else:
    print("No conflicts found.")

print("\nFINAL TEAM RANKINGS")
print("-------------------")
ranking = sorted(points.items(), key=lambda item: (-item[1], item[0]))
for rank, (team, total_points) in enumerate(ranking, start=1):
    print(f"Rank {rank}: {team} - {total_points} points")
```

8. Test Cases

```
Test Case 1: Match 1 and Match 3 overlap in time and share Tigers; conflict expected.
Test Case 2: Match 1 and Match 4 overlap at Ground A; venue conflict expected.
Test Case 3: A team wins a match; it receives 3 points.
```

```
Test Case 4: A match ends in a draw; each team receives 1 point.
Test Case 5: Teams are ranked from highest to lowest total points.
```

9. Sample Output

```
SCHEDULING CONFLICTS
--------------------
Match 1 and Match 3: same team
Match 1 and Match 4: same venue
Match 2 and Match 3: same team
Match 2 and Match 4: same team

FINAL TEAM RANKINGS
-------------------
Rank 1: Bears - 3 points
Rank 2: Eagles - 3 points
Rank 3: Tigers - 3 points
Rank 4: Lions - 2 points
Rank 5: Wolves - 1 points
Rank 6: Hawks - 0 points
```

10. Conclusion

The Multi-Event Sports Scheduling and Scoring program uses lists and tuples to store event schedules and match results. It detects overlapping events involving the same team or venue, calculates points based on match outcomes, and produces final team rankings. The program demonstrates lists, tuples, loops, conditional statements, dictionaries, sorting, and tuple unpacking.